COMPUTER NETWORKS · SEMESTER 1 · LECTURE 2

Architecture and performance: layers, reference models and the numbers behind delay

Week 2 teaching pack: lecture notes, seminar, Linux laboratory and Windows laboratory (two hours each)

Associate Professor, PhD, Nicolaie Popescu-Bodorin, 2026,
+40722882471, bodorin(on)live.com

Part	Contents	Duration
A	Lecture 2 — Architecture and performance: layers, reference models and the numbers behind delay	2 h
B	Seminar 2 — Headers, queues and throughput	2 h
C	Laboratory 2 (Linux) — Packet analysis with Wireshark and tcpdump	2 h
D	Laboratory 2 (Windows) — Packet analysis with Wireshark and pktmon	2 h

Part A · Lecture 2

After this lecture you should be able to

- distinguish a layer's service, its interface and its protocol, and explain peer communication;
- follow a message through encapsulation and demultiplexing, naming the header field that is used at each step;
- describe the seven OSI layers, compare them with the TCP/IP model and explain why IP is the "narrow waist";
- say who makes Internet standards and how an RFC becomes a standard;
- use traffic intensity to predict queueing delay, and explain why delay explodes near full load;
- compute end-to-end throughput and the bandwidth–delay product of a path;
- state the end-to-end argument and apply it to a design question.

Plan of the two hours

Part	Topic	Time
Hour 1		
1	Recap of Lecture 1 and a question to start	5 min
2	Why layers: services, interfaces, protocols	15 min
3	Encapsulation and demultiplexing in detail	15 min
4	Reference models: OSI and TCP/IP; the narrow waist	15 min
	Break	10 min

Part	Topic	Time
Hour 2		
5	Who makes the rules: standards and RFCs	8 min
6	Queueing delay and loss in depth	17 min
7	Throughput and the bandwidth–delay product	10 min
8	The end-to-end argument	10 min
9	How ping and traceroute measure all this; summary	5 min

1. Where we are

Last week we took the Internet apart into hosts, links and packet switches, saw that it is a network of independently run networks, met the four components of delay, and saw — in one table — that the work is divided into five layers. Today we look closely at that division and at the numbers. Two questions run through the lecture.

- **How is the work divided?** Why layers, which layers, and what exactly passes between them.
- **How well does it work?** What determines delay, loss and throughput, and why a network can go from fast to unusable with only a small increase in traffic.

> **Question to start**
> You download a file over a path whose links run at 1 Gbit/s, 10 Gbit/s and 100 Mbit/s. Your neighbour's path runs at 1 Gbit/s throughout, but the round-trip time is 300 ms instead of 10 ms. Who finishes a 1 MB download first? Who finishes a 10 GB download first? Keep your guesses; we answer them in section 7.

2. Why layers

A large system is understood and built by dividing it into parts with clear boundaries. Networks are divided **vertically**: each layer uses what the layer below offers and adds something, so that the layer above receives a better service. Three words must be kept apart.

Term	Meaning	Example (transport layer)
Service	*What* a layer offers to the layer above.	Reliable, ordered delivery of a byte stream between two processes.
Interface	*How* the layer above asks for the service: the operations and their parameters.	The socket calls `connect`, `send`, `recv`, `close`.
Protocol	*How* the layer implements the service, by exchanging messages with its counterpart on another machine.	TCP: segments with sequence numbers, acknowledgements, retransmissions.

The separation is what makes change possible. A service and its interface can stay fixed for decades while the protocol under them is replaced. Web browsers used the same socket interface when TCP gained new congestion-control algorithms; most of them did not need to change.

2.1 Peer communication

Each layer on one machine behaves as if it talked directly to the **same layer** on the other machine — its *peer*. A transport entity in Bucharest "sends a segment" to a transport entity in Frankfurt, and its protocol is defined entirely in terms of that conversation. In reality nothing travels horizontally except at the bottom: the segment goes down through the layers, crosses the physical medium, and comes up again on the far side. The horizontal conversation is **logical**; the vertical path is **real**.

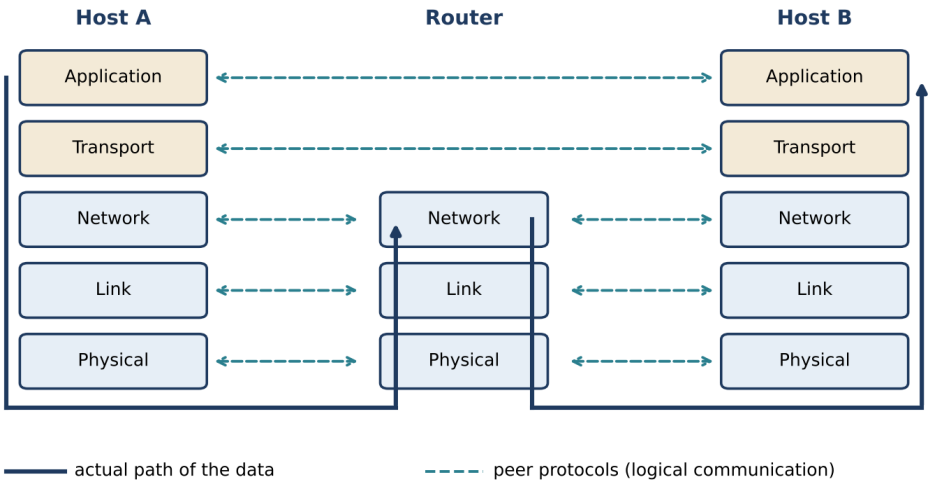


Figure 1. Peer protocols are logical; the data actually travel down, across the physical links, and up. The router implements only the three lower layers.

Note in Figure 1 that the application and transport protocols run only between the two hosts: routers do not take part in them. Network, link and physical protocols run between neighbours, hop by hop.

2.2 The price of layering

- **Overhead.** Every layer adds a header; small messages pay heavily (Seminar 2 puts numbers on it).
- **Duplicated functions.** Error detection appears at the link layer *and* in transport; sometimes both are needed, sometimes not (section 8).
- **Hidden information.** A layer cannot see what it would sometimes like to know. TCP cannot tell whether a packet was lost to congestion or to radio interference, because that knowledge lives in the link layer.

Engineers sometimes break layering on purpose ("cross-layer design"), for example to let a video application adapt to the radio conditions of a phone. The default, though, is to respect the boundaries.

3. Encapsulation and demultiplexing

At the sender each layer takes the unit it receives from above, treats it as opaque **payload** (in OSI language, a *service data unit*), and adds its own header to form its own *protocol data unit*: a segment, then a datagram, then a frame. At the receiver the reverse happens, and at every step the receiving layer must decide **which entity above it** should get the payload. That decision is **demultiplexing**, and each header carries a field for exactly this purpose.

Layer receiving the unit	Field it reads to choose the next layer	Typical values
Link (Ethernet)	**EtherType** (2 bytes)	0x0800 = IPv4, 0x86DD = IPv6, 0x0806 = ARP
Network (IPv4 / IPv6)	**Protocol** (IPv4) / **Next Header** (IPv6)	1 = ICMP, 6 = TCP, 17 = UDP
Transport (TCP / UDP)	**Destination port** (2 bytes)	53 = DNS, 80 = HTTP, 443 = HTTPS

Port numbers 0–1023 are the *well-known* ports of standard services, 1024–49151 are *registered* for particular applications, and 49152–65535 are *dynamic* ports that the operating system assigns to clients. All these numbers — EtherTypes excepted, which belong to the IEEE — are kept in public registries by IANA, the Internet Assigned Numbers Authority.

3.1 A web request, header by header

A browser sends a small HTTP request of 400 bytes to a web server. Here is what leaves the network card:

Piece	Size (bytes)	Added by	Contains, among other things
Ethernet header	14	link layer	destination MAC, source MAC, EtherType 0x0800
IPv4 header	20	network layer	source and destination IP, TTL, Protocol = 6, header checksum
TCP header	20 (+ options)	transport layer	source port (e.g. 51234), destination port 80, sequence and acknowledgement numbers
HTTP request	400	application	`GET /index.html HTTP/1.1` and header lines
Ethernet trailer	4	link layer	frame check sequence (CRC-32)

The frame is 458 bytes; 58 of them are headers. One more fact explains a number you will see everywhere: Ethernet carries at most **1500 bytes** of payload per frame, its **MTU** (maximum transmission unit). That is why a TCP segment over Ethernet typically carries at most 1500 − 20 − 20 = 1460 bytes of data.

> **Where the destination MAC address points**
> The destination *IP* address in the packet is the web server's, all the way. The destination *MAC* address is only the **next hop on this link** — usually your default gateway. Every router strips the old frame and builds a new one with new MAC addresses, while the IP header travels end to end (with the TTL decreased by one). You will verify this yourself in Laboratory 2.

4. Reference models

4.1 The OSI model

In the late 1970s the International Organization for Standardization (ISO) set out to define a complete, vendor-neutral architecture for "open systems interconnection". The resulting **OSI reference model** (ISO 7498, 1984) has seven layers:

#	OSI layer	Function	Where it lives in the Internet
7	**Application**	Services used directly by applications.	application protocols (HTTP, SMTP, DNS)
6	**Presentation**	Representation of data: encoding, compression, encryption.	inside applications and libraries (UTF-8, JSON, TLS)
5	**Session**	Dialogue control, synchronisation, checkpoints for recovery.	inside applications (HTTP sessions, TLS session resumption)
4	**Transport**	End-to-end delivery between processes.	TCP, UDP, QUIC
3	**Network**	Routing and forwarding across networks.	IP
2	**Data link**	Delivery of frames between neighbours; error detection.	Ethernet, Wi-Fi
1	**Physical**	Transmission of raw bits over a medium.	signalling on copper, fibre, radio

A mnemonic for the order from layer 1 up: “**P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way”.

4.2 The TCP/IP model

The Internet’s own architecture was never drawn so formally. RFC 1122 (1989) describes four layers: **application, transport, internet and link**. Most textbooks, including ours, split the link layer into link and physical and so use the **five-layer** model of Lecture 1. The comparison:

OSI (7)	Five-layer model (textbooks)	TCP/IP, RFC 1122 (4)
Application · Presentation · Session	Application	Application
Transport	Transport	Transport
Network	Network	Internet
Data link	Link	Link
Physical	Physical	Link

Why did TCP/IP and not OSI become the network of the world? The protocols of TCP/IP existed and ran on real machines — they shipped free with Berkeley UNIX in the early 1980s — while the OSI protocols arrived late, were complex and expensive to implement, and were designed by committee before they had been used. The OSI **model** survived as a vocabulary: engineers still say “a layer-2 switch”, “a layer-3 router”, “a layer-7 load balancer”. The OSI **protocols** did not.

4.3 The narrow waist

Draw all Internet protocols by layer and the picture looks like an hourglass (Figure 2). There are many applications and application protocols at the top and many link technologies and media at the bottom, but in the middle there is essentially **one** protocol: IP. Everything above runs over IP, and IP runs over everything below.

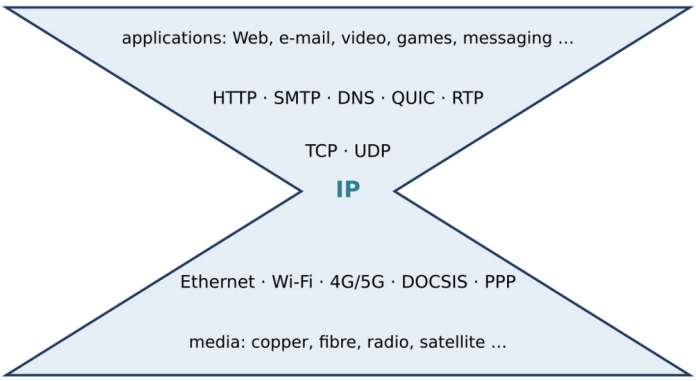


Figure 2. The hourglass: a single network-layer protocol between many applications and many link technologies.

The narrow waist is what made the Internet grow. A new application only has to run over IP to reach every network; a new link technology only has to carry IP to serve every application. Its cost is that the waist is very hard to change: moving from IPv4 to IPv6 has been under way for more than twenty-five years.

5. Who makes the rules

Body	Responsible for
IETF — Internet Engineering Task Force	Internet protocols: IP, TCP, UDP, QUIC, DNS, HTTP, TLS and many more. Open to anyone; work is done in public mailing lists and working groups.
IEEE 802 committees	Local and metropolitan networks: 802.3 Ethernet, 802.11 Wi-Fi, 802.1 bridging and VLANs.
ITU-T and **3GPP**	Telecommunications standards; 3GPP specifies the 4G and 5G mobile systems.
W3C and **WHATWG**	Web technologies: HTML, CSS, browser APIs.
IANA and the **RIRs**	Registries of numbers and names. IANA allocates address blocks to the five Regional Internet Registries; Romania is served by **RIPE NCC**, based in Amsterdam.

An Internet protocol starts as an **Internet-Draft**, which anyone may write. If a working group adopts it and reaches “rough consensus”, it is published as an **RFC**. On the standards track an RFC is first a **Proposed Standard** and may later become an **Internet Standard** (RFC 6410 reduced the levels from three to these two in 2011). RFCs are never edited after publication; a correction is a new RFC that *updates* or *obsoletes* the old one. Not all RFCs are standards: many are informational, experimental or historical — and a few, published on 1 April, are jokes.

The IETF’s working style was summarised by David Clark in 1992: “We reject kings, presidents and voting. We believe in rough consensus and running code.” A specification is taken seriously when there are independent implementations that work together.

6. Queueing delay and loss in depth

Of the four delay components, three are easy: processing is small, transmission is *L/R*, propagation is *d/s*. Queueing delay is the interesting one, because it depends on everyone else's traffic and changes from one packet to the next.

6.1 Traffic intensity

Consider one outgoing link of rate *R* bit/s. Packets of *L* bits arrive for it at an average of *a* packets per second. Bits arrive at an average rate of *La* bit/s and can leave at *R* bit/s. The ratio

$$\rho = La / R$$

is the **traffic intensity** (or utilisation) of the link. It is the single most useful number for reasoning about queues.

- **ρ > 1.** Bits arrive faster than they can leave. The queue grows without limit, and since buffers are finite, packets are lost. No buffer size fixes this. *Rule one of traffic engineering: design so that ρ stays below 1.*
- **ρ ≤ 1** with perfectly regular arrivals (one packet every *L/R* seconds): no packet ever waits.
- **ρ ≤ 1** with random, bursty arrivals — the real case: packets sometimes arrive in bunches and wait. The closer ρ is to 1, the longer and more frequent the waits.

6.2 How fast does delay grow?

Queueing theory gives exact answers for idealised traffic. In the simplest model, called **M/M/1** (random independent arrivals, random packet lengths, one link), the average time a packet spends at the router — waiting plus being transmitted — is

$$d = (L / R) / (1 - \rho)$$

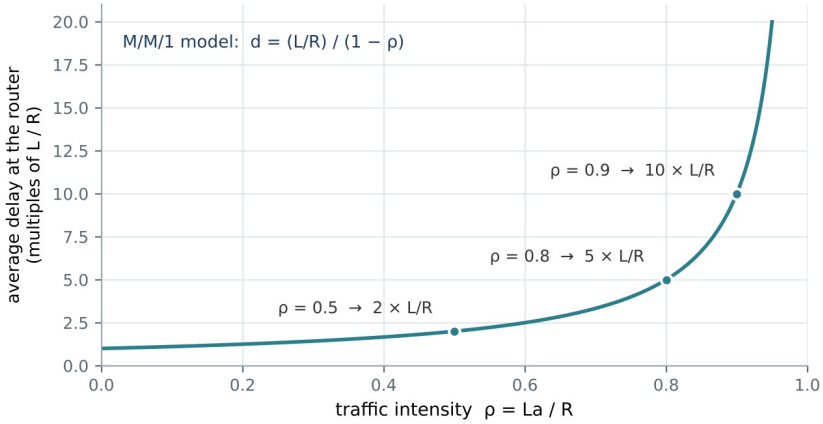


Figure 3. Average delay at a router as a function of traffic intensity, in the M/M/1 model.

Real traffic is not M/M/1, so the exact numbers should not be trusted. The **shape** of the curve should. At half load a packet takes on average twice its transmission time; at 80 % five times; at 90 % ten times; at 95 % twenty times. Going from 90 % to 95 % utilisation — a 5 % increase in traffic — doubles the delay. This is why network operators upgrade links long before they are "full", and why a network that felt fine yesterday can feel broken today.

6.3 Loss and the size of buffers

When the queue is full, arriving packets are dropped (*tail drop*). The fraction lost grows with ρ. A lost packet is retransmitted by the sender (if TCP is used) or simply missing (if not), so loss shows up to users as extra delay or as glitches.

If loss is bad, why not give routers huge buffers? Because a large buffer that is full is just a long queue: every packet waits behind it. Cheap memory led many home routers and modems to be built with very large buffers, so that a single large upload could add hundreds of milliseconds to every packet, including those of a video call. This problem was named **bufferbloat** (around 2010). The answer is not bigger or smaller buffers alone but **active queue management**: algorithms such as CoDel that drop or mark packets early, to keep the standing queue short. You will experiment with it in Semester 2.

7. Throughput and the bandwidth–delay product

7.1 End-to-end throughput

For a long transfer across a path of links with rates $R_1, R_2, \ldots, R_n$, the throughput is limited by the slowest one: **throughput = min(R_1, …, R_n)**. When a link is **shared**, what matters is the share each flow obtains. If ten transfers cross a core link of rate R, and the link divides its capacity fairly, each gets at most $R/10$. A common pattern is that the core is fast and lightly loaded and the **access links** are the bottleneck; but on a congested core link the shared capacity can become the limit.

7.2 Bandwidth–delay product

How many bits are "in flight" on a path at once — sent but not yet acknowledged? A sender transmitting at rate R over a path with round-trip time RTT has, at any moment, up to

$$\mathbf{BDP = R \times RTT}$$

bits on their way out or with their acknowledgements on the way back. This is the **bandwidth–delay product**. To keep the path full, a sender must be allowed to have at least one BDP of unacknowledged data outstanding. Example: at 1 Gbit/s with an RTT of 30 ms (roughly Bucharest–Frankfurt and back), BDP = 30 Mbit ≈ 3.75 MB. A sender limited to 64 KB outstanding — as the original TCP was — would reach only 64 KB per 30 ms ≈ 17.5 Mbit/s on that 1 Gbit/s path. We return to this when we study TCP windows.

> **Answer to the opening question**
> Small transfers are dominated by **delay**: a 1 MB download needs a few round trips to set up the connection and grow its rate, and each round trip costs 10 ms for you but 300 ms for your neighbour. You finish first.
> Large transfers are dominated by **throughput**: your bottleneck is 100 Mbit/s, so 10 GB take at least 800 s; your neighbour's path can deliver 1 Gbit/s, so — if the sender's window covers the large BDP of 1 Gbit/s × 300 ms = 37.5 MB — the transfer takes about 80 s. Your neighbour wins by a factor of ten.
> Which link is "faster" depends on the size of what you send.

8. The end-to-end argument

Where in a layered system should a function be placed — in the network, or in the end hosts? In 1984 Jerome Saltzer, David Reed and David Clark gave an answer that shaped the Internet, in a paper titled “End-to-End Arguments in System Design”.

> **The end-to-end argument (paraphrased)**
> A function can be implemented completely and correctly only with the knowledge and help of the application at the end points. Providing it inside the network is therefore never *sufficient*. It may still be useful as a **performance enhancement**, but not as a substitute for the end-to-end mechanism.

Their example is **careful file transfer**. Suppose every link checks and retransmits its frames, so each hop is “reliable”. Can the receiving host then skip checking the file? No: the file can still be corrupted inside a router’s memory, by a bug in a disk driver, or by a crash halfway through. Only an end-to-end check — a checksum computed by the sender over the file and verified by the receiver — catches all of these. And once the end-to-end check exists, perfect hop-by-hop reliability is not needed for correctness.

Hop-by-hop mechanisms are still worth having where they improve performance. Wi-Fi retransmits lost frames itself, within microseconds, because radio losses are frequent and leaving their recovery to TCP, end to end, would be much slower. The argument does not forbid the network from helping; it says the network cannot be **relied upon**.

The consequences are everywhere in the Internet’s design: a simple, best-effort IP core; reliability, ordering and congestion control in TCP at the hosts; encryption (TLS) end to end rather than link by link. Devices that break the principle — NAT boxes, firewalls, transparent proxies that inspect or modify traffic — work, but they make the network harder to change. A new transport protocol that such boxes do not recognise is often blocked. That is one reason QUIC runs over UDP and encrypts almost all of its headers.

9. How the tools measure delay

`ping` sends an ICMP *Echo Request* (type 8) and the target answers with an *Echo Reply* (type 0). The time between them is one RTT, including transmission, propagation, queueing and processing in both directions.

`traceroute` exploits the **TTL** (*time to live*) field of the IP header. Every router decreases it by one and discards a packet whose TTL reaches zero, sending back an ICMP *Time Exceeded* message (type 11). Traceroute sends probes with TTL = 1, 2, 3, …; the probe with TTL = *k* expires at the *k*-th router, which thereby reveals its address. Linux traceroute uses UDP probes to unlikely high ports (from 33434 upward) and recognises the destination by its ICMP *Port Unreachable* reply; Windows tracert uses ICMP Echo Requests and recognises the destination by its Echo Reply. Van Jacobson wrote the first traceroute in 1987 — a tool built entirely from a side effect of a header field meant for something else.

Two cautions. The return path of each reply may differ from the forward path, so an RTT does not say where the delay lies. And routers generate ICMP replies slowly and with low priority, which is why a middle hop can look slower than the destination.

Summary

- A layer offers a **service** through an **interface** and implements it with a **protocol** run between **peers**. Peer communication is logical; data travel down, across and up.
- Each layer adds a header (**encapsulation**) and uses a header field to choose the receiver above (**demultiplexing**): EtherType, IP Protocol, destination port.
- The **OSI** model has seven layers; the **TCP/IP** model four (or five in textbooks). OSI survives as vocabulary; TCP/IP as the running system. IP is the **narrow waist**.
- Internet standards are written by the **IETF** as **RFCs**; numbers are registered by **IANA**; Romania's addresses come through **RIPE NCC**.
- **Traffic intensity** ρ = La/R governs queueing. Above 1, queues grow without bound; below 1, delay rises steeply as ρ approaches 1. Very large buffers cause **bufferbloat**.
- Throughput is set by the **bottleneck**, including shares of shared links. The **bandwidth–delay product** R × RTT is how much data must be in flight to fill a path.
- The **end-to-end argument**: functions that need the endpoints' knowledge belong in the endpoints; the network may help for performance only.

Key terms

service · interface · protocol · peer · protocol data unit · payload · encapsulation · demultiplexing · EtherType · protocol number · port (well-known, registered, dynamic) · MTU · OSI model · presentation layer · session layer · TCP/IP model · narrow waist · IETF · Internet-Draft · RFC · Proposed Standard · Internet Standard · IANA · RIR · RIPE NCC · traffic intensity · M/M/1 · tail drop · bufferbloat · active queue management · bottleneck · bandwidth–delay product · end-to-end argument · ICMP Echo · TTL · Time Exceeded

Review questions

1. Give an example, other than the transport layer, of a service, an interface and a protocol at the same layer.
2. In Figure 1, which protocols does the router take part in? Which ones does it not see at all?
3. A host receives a frame. List, in order, the three header fields it reads to deliver the payload to a web browser.
4. Why does the destination MAC address change at every hop while the destination IP address does not?
5. Which OSI layers have no separate counterpart in the TCP/IP model, and where did their functions go?
6. Why does a "narrow waist" make it easy to add new applications but hard to change IP itself?
7. A link is used at 60 % on average. Its operator says: "We still have 40 % spare capacity, no need to upgrade." What would you answer?
8. Why is a bigger buffer not a cure for loss?
9. State the end-to-end argument in one sentence and give one example of a function the network performs anyway, for performance.

Exercises

1. An application sends 20-byte messages over UDP (8-byte header) and IPv6 (40-byte header) on Ethernet (18 bytes of header and trailer). What fraction of each frame is application data?
2. Packets of 1000 bytes arrive at a router whose output link runs at 8 Mbit/s. At what packet arrival rate is $\rho = 1$? Using the M/M/1 formula, what is the average delay at the router for 500, 800 and 950 packets per second?
3. A path has links of 100 Mbit/s, 40 Mbit/s and 1 Gbit/s. The 1 Gbit/s link is shared fairly by 50 simultaneous long transfers. What throughput does one of them obtain?
4. What is the bandwidth–delay product of a 10 Gbit/s transatlantic path with an RTT of 80 ms, in megabytes?
5. In a traceroute with three probes per hop to a destination 14 hops away, how many probes are sent in total (assuming every probe is answered)? Which hops answer with Time Exceeded, and which with a different message?

Answers

1. Frame = 20 + 8 + 40 + 18 = 86 bytes; data fraction 20 / 86 ≈ 23 %.
2. $L/R = 8000 / 8 \times 10^6$ s = 1 ms, so $\rho = 1$ at 1000 packets/s. $\rho = 0.5, 0.8, 0.95$ give $d = 1 / (1 - \rho)$ ms = 2 ms, 5 ms and 20 ms.
3. The shared link offers 1000 / 50 = 20 Mbit/s per transfer, which is less than 40 and 100 Mbit/s, so throughput ≈ 20 Mbit/s.
4. $10^{10} \times 0.08 = 8 \times 10^8$ bits = 100 MB.
5. 14 × 3 = 42 probes. Hops 1–13 send Time Exceeded; the destination (hop 14) sends Port Unreachable (Linux traceroute) or Echo Reply (Windows tracert).

Reading and next steps

Required

- Kurose and Ross, *Computer Networking: A Top-Down Approach*, 8th edition, Chapter 1: the sections on delay, loss and throughput, and on protocol layers and their service models.

Recommended

- J. H. Saltzer, D. P. Reed, D. D. Clark, “End-to-End Arguments in System Design”, *ACM Transactions on Computer Systems* 2(4), 1984.
- Peterson and Davie, *Computer Networks: A Systems Approach*, Chapter 1 (sections on architecture and performance).
- RFC 1122, “Requirements for Internet Hosts — Communication Layers”, section 1.1 (the Internet architecture in the IETF’s own words).
- J. Gettys, K. Nichols, “Bufferbloat: Dark Buffers in the Internet”, *Communications of the ACM*, 2012.

Laboratory 2 (this week)

Packet capture with Wireshark. You will capture your own web request, open every header of Figure 1, check which MAC and which IP addresses appear, and measure an RTT from the TCP handshake.

Next lecture

The application layer: application architectures, sockets and the services applications need, and the Web with HTTP/1.1.

PART B · WEEK 2

Seminar 2: Headers, queues and throughput

Two-hour problem session (2 × 50 minutes) accompanying Lecture 2

This seminar practises the three skills of Lecture 2: reading headers, reasoning about queues, and finding the bottleneck. Work in pairs; one pair presents each problem. Bring a calculator.

By the end of the seminar you should be able to

- place a network function in the right layer of both reference models;
- decode an Ethernet/IPv4/UDP frame from a hexadecimal dump;
- compute protocol overhead and the maximum useful throughput of a link;
- predict queueing delay from traffic intensity and use Little's law;
- find the throughput of a flow on a path with shared links, and the bandwidth–delay product;
- apply the end-to-end argument to a design proposal.

Plan

Step	Activity	Time
A	Problem 1: which layer?	10 min
B	Problem 2: decoding a frame by hand	25 min
C	Problem 3: how much of the link is useful?	15 min
	Break	10 min
D	Problems 4–5: queues — traffic intensity and Little's law	20 min
E	Problems 6–7: bottlenecks and the bandwidth–delay product	15 min
F	Problem 8: the end-to-end argument (discussion)	10 min
	Homework	(in F)

A. Which layer?

Problem 1

For each function, give the layer in the five-layer model and in the OSI model.

	Function
a	Choosing the outgoing link for a packet addressed to another country
b	Converting a document from one character encoding to another
c	Retransmitting a segment whose acknowledgement did not arrive
d	Representing a 1 as a voltage transition on a cable
e	Detecting that a frame was corrupted on a Wi-Fi link
f	Resuming a long file download from where it stopped after a disconnection
g	Delivering an incoming segment to the right process on a host

	Function
h	Deciding which of several hosts on a shared cable may transmit now

> **Solution**
> (a) network / network (3). (b) application / presentation (6). (c) transport / transport (4). (d) physical / physical (1). (e) link / data link (2). (f) application / session (5) — in the Internet this is done by applications, for example with HTTP range requests. (g) transport / transport (4), using the destination port. (h) link / data link (2), in its medium access control sublayer.

B. Decoding a frame

Problem 2

A capture tool shows the following frame, in hexadecimal, 16 bytes per line; the first column is the offset. (The 4-byte Ethernet trailer is not shown, as capture tools usually omit it.)

```
0000  52 54 00 12 34 01 52 54 00 3a 7f 10 08 00 45 00
0010  00 3d 1c 46 40 00 40 11 06 d5 0a 14 03 39 0a 14
0020  00 35 c8 22 00 35 00 29 96 d2 3b 7a 01 00 00 01
0030  00 00 00 00 00 00 03 77 77 77 07 65 78 61 6d 70
0040  6c 65 03 6f 72 67 00 00 01 00 01
```

Use this layout. **Ethernet**: destination MAC (6 bytes), source MAC (6), EtherType (2). **IPv4**: version and header length (1 byte: high 4 bits = version, low 4 bits = length in 32-bit words), type of service (1), total length (2), identification (2), flags and fragment offset (2), TTL (1), protocol (1), header checksum (2), source address (4), destination address (4). **UDP**: source port (2), destination port (2), length (2), checksum (2).

1. What are the destination and source MAC addresses? What type of payload does the frame carry?
2. What are the IP version and header length? What is the total length of the IP datagram, and is it consistent with the size of the frame?
3. What are the TTL and the transport protocol? What are the source and destination IP addresses, in dotted decimal?
4. What are the source and destination ports? Which side is the client? What application protocol is this?
5. The application data start at offset 0x2a. Without knowing the DNS format in detail, what name is being looked up? (Hint: 0x77 is the ASCII code of “w”, and the bytes 03, 07, 03 are length counters.)
6. Is the destination MAC address the MAC address of the machine 10.20.0.53? How could you find out?

Solution

1. Destination 52:54:00:12:34:01, source 52:54:00:3a:7f:10. EtherType 0x0800: an IPv4 datagram.
2. 0x45: version 4, header length 5 words = 20 bytes. Total length 0x003d = 61 bytes. The frame shown has 75 bytes = 14 (Ethernet) + 61. Consistent.
3. TTL 0x40 = 64 (it has not crossed a router yet — 64 is a common initial value). Protocol 0x11 = 17 = UDP. Source 0a.14.03.39 = 10.20.3.57; destination 0a.14.00.35 = 10.20.0.53.
4. Source port 0xc822 = 51234, destination port 0x0035 = 53. The sender uses a dynamic port and the receiver the well-known DNS port, so the sender is the client, asking a DNS server. UDP length 0x0029 = 41 = 8 + 33 bytes of DNS data.
5. After 12 bytes of DNS header comes 03 “www” 07 “example” 03 “org” 00: the name www.example.org. The final 00 01 00 01 means: type A (an IPv4 address), class IN (Internet).
6. Only if the DNS server is on the same link as the client. 10.20.3.57 and 10.20.0.53 differ in the third byte; with a /24 prefix they are in different subnets, so the frame goes to the default gateway and the MAC is the gateway’s. Check with `ip neigh` / `arp -a` — exactly what you do in Laboratory 2.

C. How much of the link is useful?

Problem 3

On the wire, every Ethernet frame is also preceded by an 8-byte preamble and followed by a 12-byte inter-frame gap, during which nothing else can be sent. A frame carries at most 1500 bytes of payload and 18 bytes of header and trailer.

1. How many byte-times does one maximum-size frame occupy on the wire?
2. A long TCP transfer over IPv4 fills every frame (20-byte IPv4 header, 20-byte TCP header). What is the maximum rate of application data on a 100 Mbit/s Ethernet?
3. Repeat with IPv6 (40-byte header).
4. A voice application sends 20-byte audio samples over UDP/IPv4, one per frame. What fraction of the link time carries audio? (Ethernet pads payloads shorter than 46 bytes up to 46.)

Solution

1. 8 + 14 + 1500 + 4 + 12 = 1538 byte-times.
2. Data per frame = 1500 − 20 − 20 = 1460 bytes; 1460 / 1538 × 100 ≈ 94.9 Mbit/s. This is why speed tests on “100 Mbit/s” links never show 100.
3. 1440 / 1538 × 100 ≈ 93.6 Mbit/s.
4. Payload = 20 + 8 + 20 = 48 bytes (above 46, so no padding). Wire time = 8 + 14 + 48 + 4 + 12 = 86 byte-times; audio fraction 20 / 86 ≈ 23 %. Real voice codecs therefore send larger samples less often, at the price of a little extra delay.

D. Queues

Problem 4 — traffic intensity

A router’s output link runs at 10 Mbit/s; packets are 1250 bytes long. Assume M/M/1 behaviour.

1. What is L/R? At what arrival rate (packets/s) does ρ reach 1?
2. Complete the table for arrival rates of 500, 800, 900, 950 and 990 packets/s: ρ and the average delay at the router.

3. Traffic grows by 5 % a month. The link is at 80 % today. In how many months does the average delay double? When will it be ten times today's value?

> **Solution**
> 1. L/R = 10 000 / 10^7 s = 1 ms. ρ = 1 at 1000 packets/s.
> 2. ρ = 0.5, 0.8, 0.9, 0.95, 0.99 → d = 2, 5, 10, 20, 100 ms.
> 3. Today d = 5 ms. Doubling to 10 ms requires ρ = 0.9, i.e. growth by a factor 0.9/0.8 = 1.125: $1.05^n = 1.125$ gives n ≈ 2.4, so in less than three months. Ten times (50 ms) requires ρ = 0.98: 0.98/0.8 = 1.225, n ≈ 4.2 months. The operator who waits for "full" waits too long.

Problem 5 — Little's law

Little's law states that, in any stable system, the average number of items inside equals the arrival rate times the average time each spends inside: **N = λ × T**. It holds for any arrival pattern.

1. A router's buffer holds on average 40 packets; packets arrive at 2000 per second. What is the average time a packet spends in the buffer?
2. A web server handles 300 requests per second, each taking on average 50 ms. How many requests are being processed at any moment, on average?
3. A TCP connection keeps 64 KB of unacknowledged data outstanding and the RTT is 20 ms. What is its throughput? (Hint: data in flight = rate × time in flight.)

> **Solution**
> 1. T = N / λ = 40 / 2000 s = 20 ms.
> 2. N = 300 × 0.05 = 15 requests.
> 3. Rate = N / T = 65 536 bytes / 0.02 s ≈ 3.3 MB/s ≈ 26 Mbit/s. The same reasoning is behind the bandwidth–delay product.

E. Bottlenecks

Problem 6 — shared core

Ten servers each send a long file to ten different clients. Every server is connected by an access link of 20 Mbit/s, every client by an access link of 8 Mbit/s, and all ten transfers cross one core link of rate R, shared fairly.

1. What is the throughput of each transfer if R = 1 Gbit/s?
2. And if R = 50 Mbit/s?
3. For which values of R is the core link the bottleneck?

> **Solution**
> 1. min(20, 8, 100) = 8 Mbit/s: the client access link limits. 2. min(20, 8, 5) = 5 Mbit/s: the shared core link limits. 3. When R / 10 < 8 Mbit/s, i.e. R < 80 Mbit/s.

Problem 7 — the bandwidth–delay product

1. Compute the BDP of: (a) a campus LAN, 1 Gbit/s, RTT 0.5 ms; (b) Bucharest–Frankfurt, 1 Gbit/s, RTT 30 ms; (c) a geostationary satellite link, 50 Mbit/s, RTT 600 ms.
2. On path (b), a sender may have at most 64 KB unacknowledged. What throughput can it reach? What fraction of the link is that?
3. How large must the allowed amount be to fill path (b)?

Solution

1. (a) $10^9 \times 0.0005 = 0.5$ Mbit = 62.5 KB. (b) 30 Mbit = 3.75 MB. (c) 30 Mbit = 3.75 MB — a slow satellite link holds as much data in flight as a fast terrestrial one.
2. 65 535 × 8 / 0.03 ≈ 17.5 Mbit/s, under 2 % of the link.
3. At least one BDP: 3.75 MB. The original TCP header could express at most 64 KB; the *window scale* option (RFC 7323) was added to remove this limit.

F. The end-to-end argument

Problem 8 — discussion

For each proposal, say whether it is consistent with the end-to-end argument, and why.

1. Routers should verify the checksum of every TCP segment and drop corrupted ones, so that hosts no longer need to check.
2. A mobile operator's radio link retransmits lost frames locally, a few times, before giving up.
3. To save bandwidth, an ISP's proxy recompresses all images that its customers download.
4. Banking applications encrypt their traffic end to end with TLS, even though the bank's internal network is already encrypted link by link.

Solution

1. Inconsistent as a *replacement*: corruption can also happen inside routers or hosts, which only an end-to-end check catches. It could be an optimisation, but routers do not check TCP checksums in practice, because of the cost.
2. Consistent: a performance enhancement. TCP still checks end to end; the link merely recovers frequent radio losses faster.
3. Problematic: the network changes data the endpoints did not ask it to change, and cannot know whether the application tolerates it (a medical image does not). Such proxies broke applications and are now defeated by HTTPS.
4. Consistent: only the endpoints can guarantee confidentiality between them. Link encryption protects each link, but not the data inside each intermediate device.

Homework

- Solve exercises 1–5 at the end of Lecture 2 without looking at the answers.
- Decode the first 54 bytes of a TCP SYN segment that you capture yourself in Laboratory 2, by hand, in the same way as Problem 2. Check every field against Wireshark.
- Read section 2 of the Saltzer–Reed–Clark paper (careful file transfer) and write five lines on whether the argument applies to e-mail delivery.

PART C · WEEK 2

Laboratory 2 (Linux): Packet analysis with Wireshark and tcpdump

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 2

In Lecture 2 the headers were drawings. Today you capture real packets and open them. By the end you will have followed one web request through every layer and measured a round-trip time from inside a TCP connection.

By the end of the laboratory you should be able to

- capture traffic with Wireshark and with `tcpdump`, and save and reopen captures;
- tell capture filters from display filters and write both;
- identify, in a real packet, the Ethernet, IP and TCP/UDP headers and the demultiplexing fields of Lecture 2;
- follow a complete web request: DNS lookup, TCP handshake, HTTP exchange, connection close;
- measure RTT from a TCP handshake and compare it with `ping`;
- watch traceroute work, and say what an observer can still see in encrypted HTTPS traffic.

Before you start

- **Environment.** The course VM or a Linux installation with a graphical desktop. (In WSL 2, use `tcpdump` there and open the files with Wireshark on Windows.)
- **Install and allow capture as a normal user:**

```
sudo apt install -y wireshark tshark tcpdump traceroute curl dnsutils
# answer "Yes" when asked whether non-superusers may capture packets
sudo usermod -aG wireshark $USER
# log out and in again, then check:
groups | grep wireshark
```

- **Rules of conduct.** Capture only on your own machine and only your own traffic. Capturing other people's traffic without permission is unethical and, in many situations, illegal.
- **What to hand in.** A report with answers to **Q1–Q16**, plus the capture file `web.pcapng` from Task 3.
- **Interface name.** Below, `eth0` stands for your main interface; find yours with `ip -br link` (it may be `enp0s3`, `ens33`, `wlp2s0`…).

Addresses and values shown in this sheet are illustrations. Yours will differ.

Plan

Step	Activity	Time
1	A first capture and a tour of Wireshark	15 min
2	Capture filters and display filters	10 min
3	One web request, layer by layer	25 min
	Break	10 min
4	The whole conversation: streams, statistics, flow graph	10 min

Step	Activity	Time
5	Measuring RTT from the handshake	10 min
6	Traceroute under the microscope	15 min
7	Command-line capture: tcpdump and tshark	10 min
8	HTTPS: what remains visible	5 min

Task 1 — A first capture

Start Wireshark, double-click your main interface, and in a terminal run:

```
curl -s http://example.org/ > /dev/null
```

Stop the capture (red square). Wireshark shows three panes: the **packet list** (one line per packet), the **packet details** (the headers of the selected packet, one expandable line per layer) and the **packet bytes** (the raw hexadecimal). Clicking a field in the details highlights its bytes — the link between Lecture 2's tables and reality.

- **Q1.** How many packets did you capture in total, and roughly how many of them belong to your `curl` command? What else is your machine sending without your asking?
- **Q2.** Select any packet and expand its details. Which layers does Wireshark list? Match them with the five-layer model.

Task 2 — Capture filters and display filters

There are two kinds of filters, with **different syntax**:

	Capture filter	Display filter
When it acts	before capture: unmatched packets are never recorded	after capture: unmatched packets are only hidden
Syntax	BPF, as in tcpdump	Wireshark's own field language
Examples	`host example.org` · `port 53` · `tcp port 80` · `icmp`	`dns` · `http` · `tcp.port == 80` · `ip.addr == 10.20.0.53` · `icmp` · `tcp.flags.syn == 1`

Restart the capture with the capture filter `port 53 or port 80`, repeat the `curl`, and then try the display filters of the table one by one.

- **Q3.** Write a display filter that shows only DNS responses (hint: the field `dns.flags.response`), and one that shows only packets sent *by* your machine.

Task 3 — One web request, layer by layer

Empty the DNS cache, so that the request starts with a real DNS lookup, then capture everything and save the file:

```
resolvectl flush-caches        # on systems with systemd-resolved
# start a capture in Wireshark (no capture filter), then:
curl -s http://example.org/ > /dev/null
# stop, then File → Save As… web.pcapng
```

Apply the display filter `dns or tcp.port == 80`. You should find, in order: a DNS query and its response; three TCP packets with flags SYN, SYN-ACK, ACK (the **handshake**); the HTTP GET; the response `200 OK`; and packets with the FIN flag that close the connection.

Select the HTTP GET packet and fill in this table in your report:

Layer	Field	Value in your capture
Ethernet	Source MAC / Destination MAC	
Ethernet	Type	
IPv4	Source / Destination address	
IPv4	TTL · Protocol · Total length	
TCP	Source port / Destination port	
TCP	Sequence number · Header length	
HTTP	Request line · Host header	

- **Q4.** Which of the values in the table are the demultiplexing keys of Lecture 2? What would happen if the EtherType were 0x86DD?
- **Q5.** Compare the destination MAC address of the GET with your gateway's MAC (`ip neigh`). Explain the result.
- **Q6.** Look at the TTL of the server's *reply*. Estimate the number of routers between you and the server. Why could the forward and return paths have different lengths?
- **Q7.** How many bytes of headers precede the first byte of HTTP in the GET frame? Check against Lecture 2, section 3.1.

Task 4 — The whole conversation

- Right-click the GET packet → **Follow → TCP Stream**. Wireshark shows the application data of both directions, in colour.
- **Statistics → Protocol Hierarchy** shows how the captured bytes divide between protocols.
- **Statistics → Conversations** lists every pair of endpoints and the bytes exchanged.
- **Statistics → Flow Graph** draws the exchange as a time–sequence diagram, like the diagrams in the lectures.
- **Q8.** Draw (or paste) the flow graph of your web request and label: DNS lookup, handshake, request, response, close. How many packets did it take to fetch one small page?

Task 5 — Measuring RTT from the handshake

Set **View → Time Display Format → Seconds Since Previous Displayed Packet**. Apply the display filter `tcp.flags.syn == 1`, which shows the SYN and the SYN-ACK.

- **Q9.** The time between your SYN and the server's SYN-ACK is one RTT. What is it? Now run `ping -c 5 example.org` and compare. Explain any difference. (Wireshark also computes it for you: look for the field `tcp.analysis.ack_rtt` under *[SEQ/ACK analysis]* of the ACKs.)
- **Q10.** In the DNS exchange, how long did the resolver take to answer? Repeat the `curl` without flushing the cache: is there still a DNS lookup? Why?

Task 6 — Traceroute under the microscope

Capture with the display filter `icmp or (udp and ip.dst == <address of example.org>)` while running a short traceroute:

```
traceroute -n -q 1 -m 6 example.org
```

- **Q11.** Find the outgoing probes. What protocol and destination ports do they use? What is the TTL of the first, second, third probe?
- **Q12.** Find the replies. What ICMP type and code do the routers send back? Open one: inside the ICMP message, Wireshark shows the beginning of *your* original probe. Why is it useful for the ICMP message to carry it?

Task 7 — Command-line capture

On servers there is no graphical interface. The same captures are made with `tcpdump` (and analysed with `tshark`, Wireshark's command-line twin):

```
sudo tcpdump -i eth0 -n -c 20 'port 53 or port 80'
sudo tcpdump -i eth0 -n -w cli.pcap 'host example.org' &
curl -s http://example.org/ > /dev/null; sudo pkill tcpdump
tshark -r cli.pcap
tshark -r cli.pcap -Y http.request -T fields -e ip.src -e ip.dst -e http.host -e
http.request.uri
```

- **Q13.** Open `cli.pcap` in Wireshark. Is it the same conversation as in Task 3? What is the difference between the `-n` option and its absence in `tcpdump`'s output?

Task 8 — HTTPS: what remains visible

Capture while running `curl -s https://example.org/ > /dev/null` and apply the display filter `tls`.

- **Q14.** Can you read the HTTP request? In the first TLS message (*Client Hello*), find the extension **server_name**. What does it contain?
- **Q15.** List what an observer on your network can still learn about this HTTPS visit (think of addresses, names, sizes and times).
- **Q16.** Which layers of Lecture 2 are encrypted by TLS, and which travel in clear? Relate your answer to the end-to-end argument.

If something does not work

Symptom	Likely cause and what to try
Wireshark lists no interfaces	You are not yet in the `wireshark` group. Run `groups`; log out and in again.
No DNS query appears	The answer was cached. Flush the cache (`resolvectl flush-caches`) or query another name.
No HTTP, only TLS	The site redirected you to HTTPS. Use `http://example.org/` exactly, or `curl` without `-L`.
The capture is flooded with other traffic	Use a capture filter, or a display filter on your server's address.
`tcpdump: permission denied`	Prefix with `sudo`, or use Wireshark's group permission.

PART D · WEEK 2

Laboratory 2 (Windows): Packet analysis with Wireshark and pktmon

Two-hour laboratory sheet (2 × 50 minutes) accompanying Lecture 2

This sheet does on Windows what the Linux sheet does on Linux. Wireshark itself is the same program on both systems; what differs is how packets are captured underneath, the tools that generate the traffic, and the built-in command-line capture tool.

By the end of the laboratory you should be able to

- capture traffic with Wireshark on Windows and with the built-in `pktmon`;
- tell capture filters from display filters and write both;
- identify the Ethernet, IP and TCP/UDP headers and the demultiplexing fields in real packets;
- follow a complete web request and measure RTT from the TCP handshake;
- explain how Windows `tracert` differs from Linux `traceroute`, from the packets themselves;
- say what an observer can still see in HTTPS traffic.

Before you start

- **Install Wireshark** from wireshark.org if the laboratory image does not have it. The installer also installs **Npcap**, the driver that lets Windows programs capture packets; accept it. Installation requires administrator rights; capturing afterwards does not.
- **Terminal.** Use PowerShell. Some commands are marked *(admin)*: open the terminal with "Run as administrator" for those.
- **Wi-Fi note.** On a Wi-Fi adapter, Windows hands Wireshark frames that *look like* Ethernet frames: the real 802.11 headers are not shown unless the adapter supports monitor mode. The same is true on Linux in normal mode. Everything in this sheet works either way.
- **Rules of conduct and what to hand in** are as in the Linux sheet: answers to **Q1–Q16** and the file `web.pcapng` from Task 3.

Addresses and values shown in this sheet are illustrations. Yours will differ.

Plan

Step	Activity	Time
1	A first capture and a tour of Wireshark	15 min
2	Capture filters and display filters	10 min
3	One web request, layer by layer	25 min
	Break	10 min
4	The whole conversation: streams, statistics, flow graph	10 min
5	Measuring RTT from the handshake	10 min
6	tracert under the microscope	15 min
7	Command-line capture: pktmon and tshark	10 min

Step	Activity	Time
8	HTTPS: what remains visible	5 min

Task 1 — A first capture

Start Wireshark and double-click the adapter that carries traffic (the one with a moving activity line: usually *Ethernet* or *Wi-Fi*). In PowerShell run:

```
curl.exe -s http://example.org/ -o NUL
```

Stop the capture. Identify the three panes — packet list, packet details, packet bytes — and click a field in the details to see its bytes highlighted.

- **Q1.** How many packets did you capture, and roughly how many belong to your command? Windows machines are talkative: name three other protocols you see (look at the *Protocol* column).
- **Q2.** Expand the details of any packet. Match the layers Wireshark lists with the five-layer model.

Task 2 — Capture filters and display filters

	Capture filter	Display filter
When it acts	before capture: unmatched packets are never recorded	after capture: unmatched packets are only hidden
Syntax	BPF, as in tcpdump	Wireshark's own field language
Examples	`host example.org` · `port 53` · `tcp port 80` · `icmp`	`dns` · `http` · `tcp.port == 80` · `ip.addr == 10.20.0.53` · `icmp` · `tcp.flags.syn == 1`

Restart with the capture filter `port 53 or port 80`, repeat the `curl.exe`, and try the display filters one by one.

- **Q3.** Write a display filter that shows only DNS responses, and one that shows only packets sent by your machine (find your address with `ipconfig`).

Task 3 — One web request, layer by layer

Clear the DNS cache so the request starts with a real lookup, capture without a filter, and save:

```
ipconfig /flushdns
# start a capture in Wireshark, then:
curl.exe -s http://example.org/ -o NUL
# stop, then File → Save As… web.pcapng
```

With the display filter `dns or tcp.port == 80`, find: DNS query and response; the handshake (SYN, SYN-ACK, ACK); the HTTP GET; the `200 OK`; the closing FIN packets. Select the GET and fill in the table:

Layer	Field	Value in your capture
Ethernet	Source MAC / Destination MAC	
Ethernet	Type	

Layer	Field	Value in your capture
IPv4	Source / Destination address	
IPv4	TTL · Protocol · Total length	
TCP	Source port / Destination port	
TCP	Sequence number · Header length	
HTTP	Request line · Host header	

- **Q4.** Which values are the demultiplexing keys of Lecture 2?
- **Q5.** Compare the destination MAC of the GET with your gateway's MAC (`arp -a` or `Get-NetNeighbor`). Explain.
- **Q6.** What TTL does your machine put in outgoing packets? Compare with the TTL in the server's reply, and estimate the number of routers between you.
- **Q7.** How many bytes of headers precede the first byte of HTTP? Check against Lecture 2, section 3.1.

Task 4 — The whole conversation

Use **Follow → TCP Stream**, **Statistics → Protocol Hierarchy**, **Statistics → Conversations** and **Statistics → Flow Graph**, as in the Linux sheet.

- **Q8.** Paste the flow graph of your request and label: DNS lookup, handshake, request, response, close. How many packets did one small page take?

Task 5 — Measuring RTT from the handshake

Set **View → Time Display Format → Seconds Since Previous Displayed Packet** and apply `tcp.flags.syn == 1`.

- **Q9.** What is the time between SYN and SYN-ACK? Compare with `ping -n 5 example.org` and with the field `tcp.analysis.ack_rtt`. Explain any difference.
- **Q10.** How long did the DNS server take to answer? Repeat the `curl.exe` without flushing: is there still a DNS lookup? Look at `ipconfig /displaydns` to see why.

Task 6 — tracert under the microscope

Capture with the display filter `icmp` while running:

```
tracert -d -h 6 -w 1000 example.org
```

- **Q11.** What kind of packets does `tracert` send? What is their TTL, hop after hop, and how many are sent per hop? How does this differ from Linux `traceroute` (Lecture 2, section 9)?
- **Q12.** What ICMP type do the routers send back? Open one: what does Wireshark show inside the ICMP message, and why is it useful?

Task 7 — Command-line capture: pktmon and tshark

Windows has a built-in packet monitor, **pktmon**, which needs an administrator terminal. It writes its own format, which it can convert for Wireshark:

```
# (admin)
pktmon start --capture --comp nics --pkt-size 0 -f cli.etl
curl.exe -s http://example.org/ -o NUL
pktmon stop
pktmon etl2pcap cli.etl -o cli.pcapng
```

Wireshark also installs `tshark`, its command-line version:

```
& "C:\Program Files\Wireshark\tshark.exe" -D
& "C:\Program Files\Wireshark\tshark.exe" -r cli.pcapng -Y http.request -T fields -e
ip.src -e ip.dst -e http.host -e http.request.uri
```

- **Q13.** Open `cli.pcapng` in Wireshark. Is it the same conversation as in Task 3? `pktmon` sees each packet at several points inside the network stack: do you notice any packets that appear more than once?

Task 8 — HTTPS: what remains visible

Capture while running `curl.exe -s https://example.org/ -o NUL` and apply the display filter `tls`.

- **Q14.** Can you read the HTTP request? In the *Client Hello*, find the **server_name** extension. What does it contain?
- **Q15.** List what an observer on your network can still learn about this HTTPS visit.
- **Q16.** Which layers are encrypted by TLS, and which travel in clear? Relate your answer to the end-to-end argument.

Linux and Windows side by side

Purpose	Linux	Windows
Capture driver	kernel (libpcap)	Npcap
Graphical capture	Wireshark	Wireshark
Built-in command-line capture	`tcpdump`	`pktmon` *(admin)*
Command-line analysis	`tshark`	`tshark.exe`
Flush the DNS cache	`resolvectl flush-caches`	`ipconfig /flushdns`
Traceroute probes	UDP to high ports (default)	ICMP Echo Request
Typical initial TTL	64	128

If something does not work

Symptom	Likely cause and what to try
Wireshark lists no interfaces	Npcap is missing or was not installed. Re-run the Wireshark installer and select Npcap.
Capture on the right adapter shows nothing	You picked a virtual adapter (VPN, Hyper-V). Choose the one whose activity line moves.
No DNS query appears	The name was cached. Run `ipconfig /flushdns` again just before the request.
`pktmon` reports “access denied”	Open the terminal with “Run as administrator”.

Symptom	Likely cause and what to try
`curl` prints an object, not a page	You ran the PowerShell alias. Type `curl.exe`.

Next week

Lecture 3 opens the application layer with the Web and HTTP. In Laboratory 3 you write your first network programs: a TCP client and server using sockets.